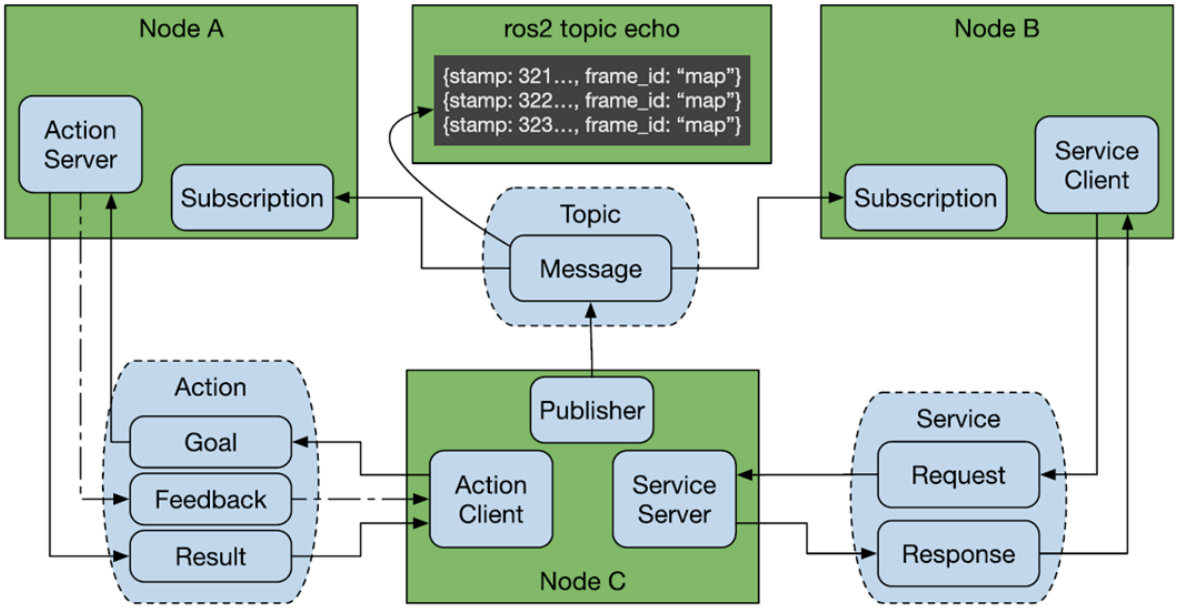
# 10. ROS 2 Custom Interface Message

## 1. Introduction to Communication Interfaces

In the ROS system, whether it is a topic, a service, or an action, an important concept is always involved—the communication interface.

Communication is not a monologue; it involves two or more parties exchanging information. What is the content of the exchange? To make it easy for everyone to understand, we can define a standard structure for the transmitted data, and this is the communication interface.

Interfaces can reduce the dependencies between programs, making it easier for us to use other people's code and for others to use our code. This is the core goal of ROS: reducing the reinvention of the wheel.




ROS has three commonly used communication mechanisms, namely topics, services, and actions. Only through the interfaces defined in each type of communication can various nodes be organically connected together.

## 2. Process of Creating a Custom Interface

The process of creating a custom interface message is similar to the process of writing an executable program in a package. The main steps are as follows:

1. Create an interface package;
2. Create and edit .msg files, .srv files, and .action files
3. Edit the configuration file;
4. Compile;
5. Test.

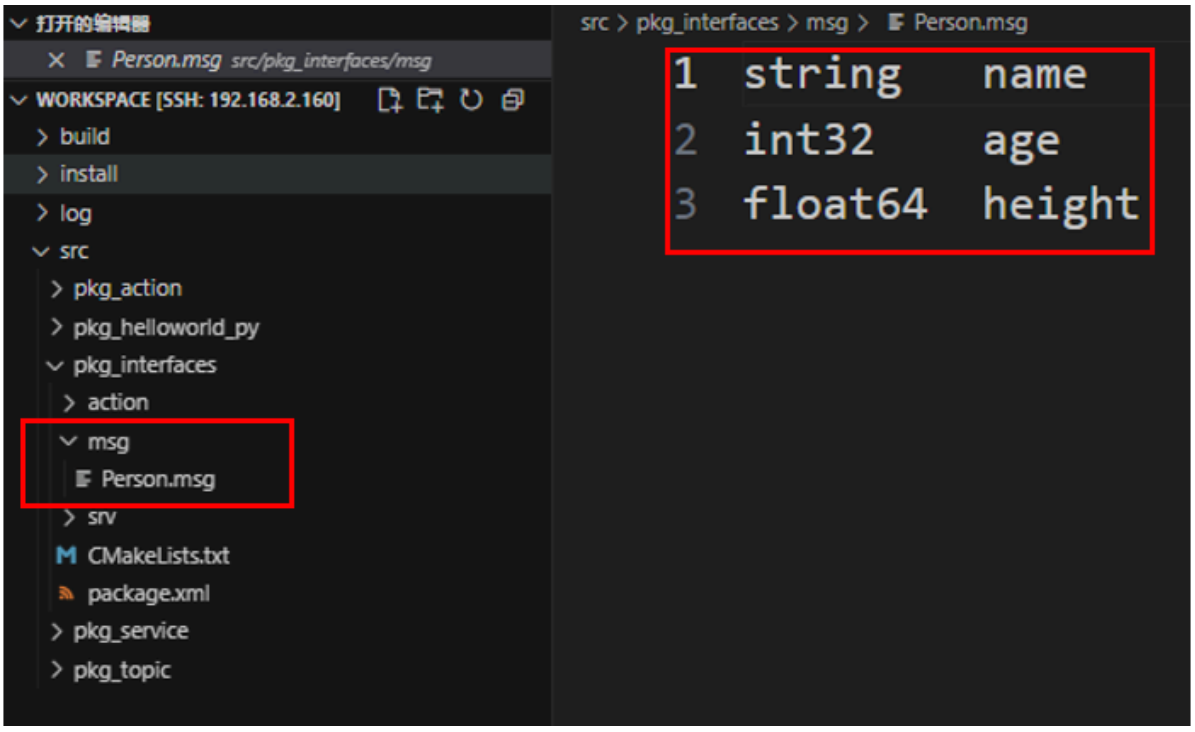
# 3. Creating a Custom Action Communication Interface

In the 【11. ROS 2 Action Communication Server Implementation】 case, we have already demonstrated the complete process of creating an action communication interface. You can go back and review it first; it will not be repeated here.

# 4. Creating a Custom Topic Communication Interface

1. In the 【ROS 2 Action Communication Server Implementation】 course, we have already created the custom interface package. Create a new msg folder under the package pkg_interfaces, create a new **Person.msg** file in the msg folder, and enter the following content in the file:

```
string   name
int32    age
float64  height
```



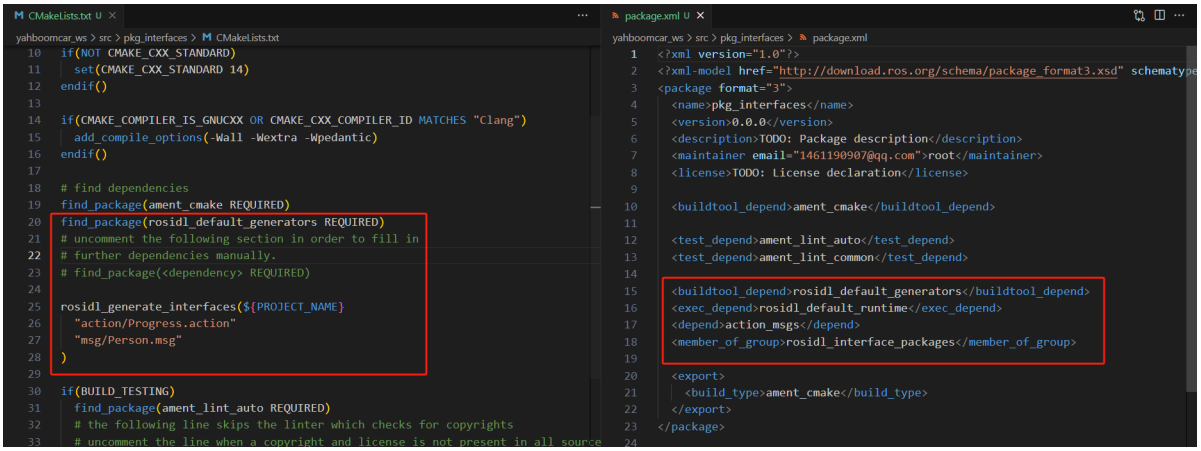

2. Add the following configuration to package.xml and CMakeLists.txt:

- CMakeLists.txt

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Progress.action"
  "msg/Person.msg"
)
```

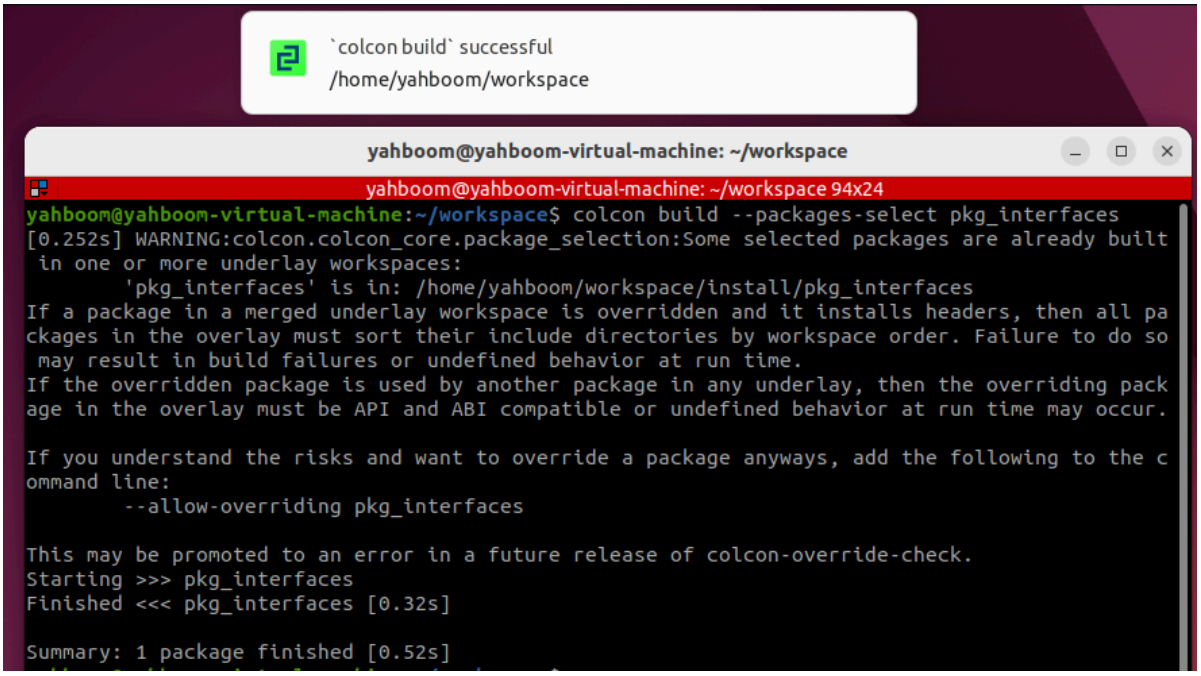
- package.xml

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<depend>action_msgs</depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```




3. In the terminal, enter the current workspace and compile the package:

```
colcon build --packages-select pkg_interfaces
```




4. Test whether the interface is normal

- Refresh the environment variables first

```
source install/setup.bash
```

- Check the interface type

```
ros2 interface show pkg_interfaces/msg/Person
```

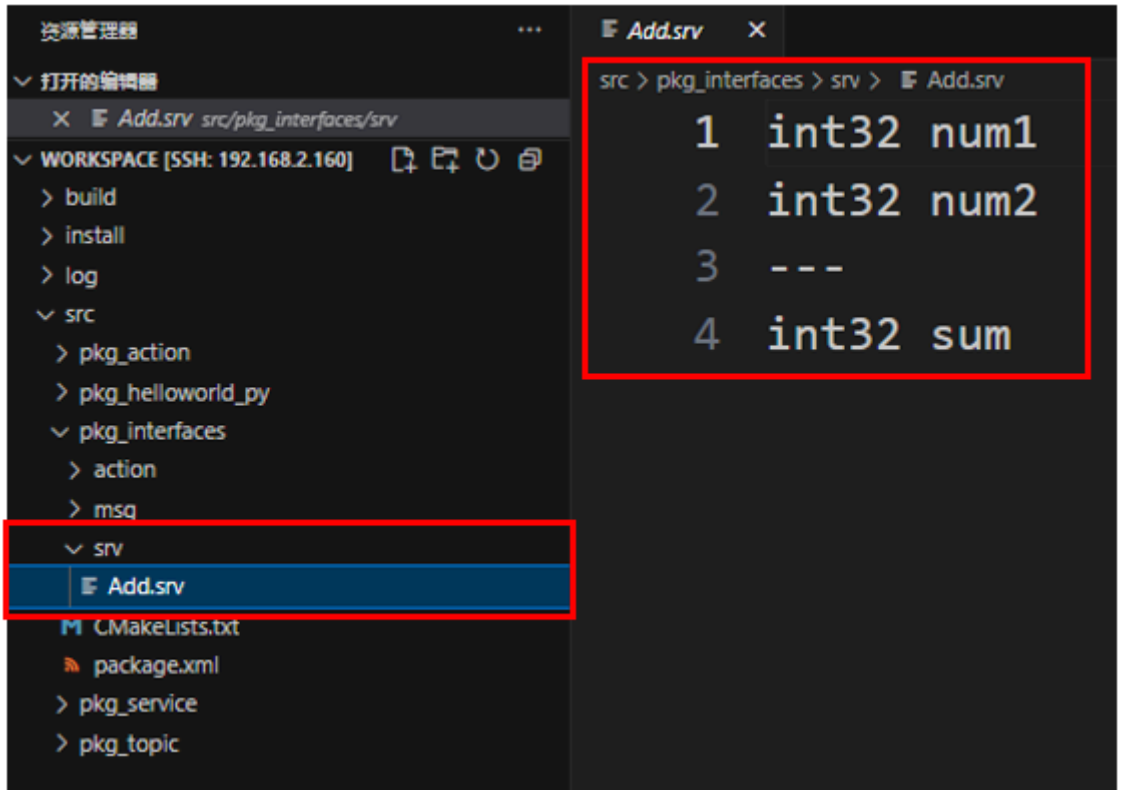
Under normal circumstances, the terminal will output content identical to the `Person.msg` file.

```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 interface show pkg_interfaces/msg/Person
string   name
int32    age
float64  height
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws#
```

# 5. Creating a Custom Service Communication Interface

1. In the 【ROS 2 Action Communication Server Implementation】 course, we have already created the custom interface package. Create a new srv folder under the package pkg_interfaces, create a new Add.srv file in the srv folder, and enter the following content in the file:

```
int32 num1
int32 num2
---
int32 sum
```



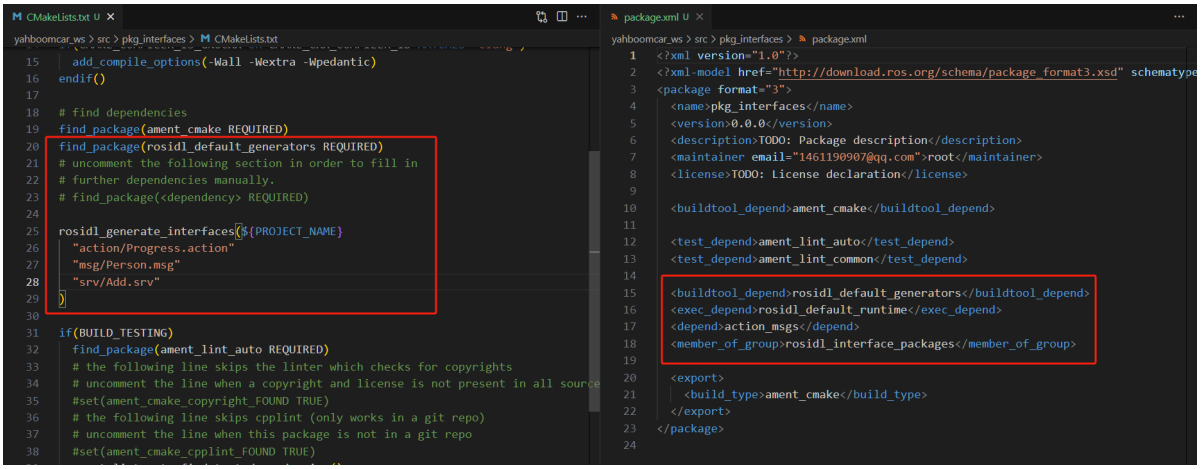

2. Add the following configuration to package.xml and CMakeLists.txt:

- CMakeLists.txt

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Progress.action"
  "msg/Person.msg"
  "srv/Add.srv"
)
```

- package.xml

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<depend>action_msgs</depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```




3. In the terminal, enter the current workspace and compile the package:

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws
colcon build --packages-select pkg_interfaces
source install/setup.bash
```

4. Test

```
ros2 interface show pkg_interfaces/srv/Add
```

Under normal circumstances, the terminal will output content identical to the `Person.msg` file.

